

GLA UNIVERSITY, MATHURA
Department of Computer Engineering & Applications
Institute of Engineering & Technology

COURSE FILE

Computer Programming (BCSG 0802)

Subject Name & Code:	Computer Programming (BCSG 0802)
Subject Matter Expert:	Mr. Ajay Ashok Dabade
Programme:	B. Tech (CSE)
Year / Semester:	1st Year / Semester I
Session:	2026-27

Index

S. No.	Context
1.	List of Subject/Faculty
2.	Subject Syllabus
3.	Evaluation Scheme & Activity Calendar
4.	Lecture Delivery Plan (Blowup)
5.	Placement-aligned Action Plan/Value Addition
6.	Scenario-Based Sample Assignment Sheets
7.	Sample Tutorial Sheets/Practice Sheets
8.	Minutes of Meetings
9.	Previous Year Question Papers

List of Subject/Faculty

S.No	Subject Code	Subject Name	Faculty Name
1	BCSG 0802	Computer Programming	Mr. Rahul Yadav
2	BCSG 0802	Computer Programming	Ms. Borada Daksha Kalubhai
3	BCSG 0802	Computer Programming	Mr. Ajay Ashok Dabade
4	BCSG 0802	Computer Programming	Ms. Robin Khurana
5	BCSG 0802	Computer Programming	Ms. Pragya Singh
6	BCSG 0802	Computer Programming	Mr. Pranshu Singh
7	BCSG 0802	Computer Programming	Ms. Ruchi Talwar
8	BCSG 0802	Computer Programming	Mr. Udayveer Singh

Subject Syllabus

BCSG 0802: COMPUTER PROGRAMMING

Objective: To provide a solid foundation in problem-solving techniques and develop robust programming skills using the C programming language. This course aims to equip students with the essential programming constructs and logic that will enable them to seamlessly transition to other programming languages and effectively tackle real-world computational problems.

Credits: 03 | L-T-P-J: 3-1-0-0

Module No.	Content	Teaching Hours
I	- Basics of C: Overview, Structure of a C program, Identifier, Keywords, Variables, Data types, Formatted Input and output. - Operators and Expression: Assignment, Unary, Arithmetic, Relational, Logical, Bitwise, Conditional, Special operators and their precedence & Associativity. - IEEE representation of data types like float & double, Lvalue and Rvalue. - Type Conversion: Type Promotion in expression, Conversion by Assignment, Truncation and Casting Arithmetic expression. - Decision and Case Control Structure: if, if-else, nested if-else, Decisions using switch, switch versus if-else ladder. - Loop Control Structure: For loop, while loop, do-while loop, nesting of loops, break, continue and goto statements. - Arrays: Introduction, one-dimensional Array-Declaration, Initialization, Address Calculation. - Operations on Arrays: Insertion, Deletion, Linear Search & Bubble Sort.	20
II	- Two-Dimensional Arrays: Declaration, Initialization, Accessing Elements, Applications in Matrices, and Problem Solving Using 2D Arrays. - Matrix Operations: Addition, Subtraction, Multiplication, and Transposition. - String: Introduction, One dimensional and two dimensional Array-Declarations, Initialization. Operations on String: Length, Copy, Reverse, Concatenate, Compare with & without built-in functions. - Functions: Declaration and Definition, Category of Functions, Parameter Passing Techniques – Call by Value, Passing Arrays to Functions. - Pointers: Declaration and Initialization of Pointer Variables, Accessing a Variable through its Pointer, Arrays and Pointers, Pointer and Strings, Pointer Arithmetic, Pointers to Pointers, Array of Pointers, Pointer to an Array, Two Dimensional Array and Pointers, Pointers to Functions, Dynamic Memory Allocation, void Pointer and Null Pointer. - Introduction to User Defined Types: enum, typedef, Union and Structure.	20

Text Books:

- K. N. King, "C Programming: A Modern Approach", 2nd Edition, W. W. Norton & Company
- MASTERING C by K R Venugopal; Sudeep R Prasad
- Paul Deitel and Harvey Deitel, "C How to Program", 8th Edition, Pearson

Reference Books:

- Peter van der Linden, "Expert C Programming: Deep C Secrets", Prentice Hall
- W. Richard Stevens, "Advanced Programming in the UNIX Environment", 3rd Edition, Addison-Wesley Professional

Outcome: After completion of the course, the student will be able to:

- CO1: Demonstrate an understanding of fundamental problem-solving techniques and algorithms.
- CO2: Apply the core principles of the C programming language to develop efficient and effective code.
- CO3: Manipulate and process data using arrays and strings, implementing basic algorithms for common operations.
- CO4: Design and utilize functions to modularize code, enhance readability, and solve complex problems.
- CO5: Implement recursive algorithms and understand their applications in problem-solving.
- CO6: Utilize pointers to efficiently manage memory and perform advanced data manipulations.
- CO7: Understand and apply file handling techniques to perform input and output operations.

- CO8: Develop and implement algorithmic solutions to a variety of computational problems, demonstrating the ability to write clean, well-documented, and maintainable code.

Mapping of Course Outcomes (COs) with Program Outcomes (POs) and Program Specific Outcomes (PSOs):

COs	POs/PSOs
CO1	PO1, PO2, PO4, PO7 / PSO1, PSO3
CO2	PO1, PO2, PO3, PO6 / PSO1, PSO3
CO3	PO1, PO2, PO4, PO6 / PSO1, PSO3
CO4	PO1, PO3, PO7 / PSO1, PSO2
CO5	PO1, PO2, PO4, PO7 / PSO1, PSO3
CO6	PO1, PO2, PO3, PO4, PO7 / PSO1, PSO2
CO7	PO1, PO3, PO5, PO7 / PSO1
CO8	PO1, PO2, PO4, PO6, PO7 / PSO1, PSO3

PO1: Engineering Knowledge | PO2: Problem Analysis | PO3: Design/Development of Solutions | PO4: Conduct Investigations of Complex Problems | PO5: The Engineer and Society | PO6: Communication | PO7: Lifelong Learning

PSO1: Software Development | PSO2: Advanced Programming | PSO3: Problem-Solving and Computational Thinking

Evaluation Scheme & Activity Calendar

Computer Programming (BCSG 0802) — Marks Distribution (100 Marks)			
Mid-Term Exam — 30 Marks			
End-Term Exam — 45 Marks			
TA — 25 Marks	Quiz-1 (5 Marks)	Quiz-2 (5 Marks)	General Performance# (10 Marks) / Attendance (5 Marks)

In the General Performance, Assignment/Case Study/Presentation are to be conducted by respective faculties in their sections.

Note: Quizzes are to be conducted by respective faculty on the GLA platform.

Sr. No.	Activity Name	Deadline
1.	Syllabus Sharing	TBD
2.	LDP Sharing	TBD
3.	Sharing of E-Books	TBD
4.	Tutorial Sheet Sharing (Module-1)	TBD
5.	Practice Sheet Sharing (Module-1)	TBD
6.	Quiz-1	TBD
7.	Assignment-1	TBD
8.	Tutorial Sheet Sharing (Module-2)	TBD
9.	Practice Sheet Sharing (Module-2)	TBD
10.	Quiz-2	TBD
11.	Assignment-2	TBD

Note: Exact activity, quiz, and assignment dates for the current session to be confirmed and updated here.

Lecture Delivery Plan (Blowup)

BCSG 0802: Computer Programming

B.Tech. Computer Science & Engineering | 3 Credits | L-T-P-J: 3-1-0-0 | 40 Teaching Hours

Prepared & Designed by: Mr. Ajay Dabade

Technical Trainer / Faculty, Department of Computer Engineering & Applications

1. Course Overview

Course Code	BCSG 0802	Course Title	Computer Programming
Programme	B.Tech. Computer Science & Engineering	Credits	03
L-T-P-J	3-1-0-0	Total Teaching Hours	40
Module I	20 Hours	Module II	20 Hours
Primary Language	C	Academic Session	2026-27

2. Course Objective

To provide a solid foundation in problem-solving techniques and develop robust programming skills using the C programming language. The course aims to equip students with essential programming constructs and logic that enable them to transition to other programming languages and tackle real-world computational problems.

3. Course Outcomes

CO	Course Outcome
CO1	Demonstrate an understanding of fundamental problem-solving techniques and algorithms.
CO2	Apply the core principles of the C programming language to develop efficient and effective code.
CO3	Manipulate and process data using arrays and strings, implementing basic algorithms for common operations.
CO4	Design and utilize functions to modularize code, enhance readability, and solve complex problems.
CO5	Implement recursive algorithms and understand their applications in problem-solving.
CO6	Utilize pointers to efficiently manage memory and perform advanced data manipulations.
CO7	Understand and apply file handling techniques to perform input and output operations.
CO8	Develop and implement algorithmic solutions to a variety of computational problems, demonstrating the ability to write clean, well-documented, and maintainable code.

4. Lesson Delivery Plan

Lec. No.	Mod.	Topic	Sub-topic	Learning Objective	Teaching Method	Proposed Activity	Expected Outcome	CO	Bloom's	Assessment	Ref.
1	I	Basics of C	Overview of C, applications and features	Understand the purpose, characteristics and applications of C programming	Interactive Lecture	Discuss real-world applications of C and identify problems that can be solved using programming	Students can explain the role and applications of C programming	CO1, CO2	Understand	Diagnostic questioning / Oral quiz	R1, R3
2	I	Basics of C	Structure of a C Program	Understand the basic structure and components of a C program	Lecture + Live Coding	Analyze a simple C program and identify header, main(), declarations and statements	Students can identify and construct the basic structure of a C program	CO2	Understand, Apply	Code-tracing exercise	R1, R3
3	I	Basics of C	Identifiers, Keywords and Variables	Understand naming rules and use of identifiers, keywords and variables	Interactive Lecture + Demonstration	Identify valid and invalid identifiers and create variables for a given problem	Students can correctly declare and use identifiers and variables	CO2	Understand, Apply	Short coding exercise	R1
4	I	Basics of C	Data Types	Understand basic C data types and their use	Lecture + Demonstration	Write programs using different data types and observe their values	Students can select appropriate data types for a problem	CO2	Understand, Apply	Quiz + coding task	R1, R3

Lec. No.	Mod.	Topic	Sub-topic	Learning Objective	Teaching Method	Proposed Activity	Expected Outcome	CO	Bloom's	Assessment	Ref.
5	I	Basics of C	Formatted Input and Output	Use formatted input and output operations in C	Live Coding + Guided Practice	Create a program that accepts student information and displays it in formatted form	Students can use printf() and scanf() with appropriate format specifiers	CO2	Apply	Programming exercise	R1
6	I	Operators and Expressions	Assignment, Unary and Arithmetic Operators	Apply assignment, unary and arithmetic operators to solve numerical problems	Lecture + Live Coding	Develop programs for arithmetic calculations such as simple interest and average	Students can construct arithmetic expressions correctly	CO2	Apply	Coding exercise	R1, R3
7	I	Operators and Expressions	Relational, Logical and Conditional Operators	Use relational, logical and conditional operators in expressions	Demonstration + Problem Solving	Predict outputs for compound logical expressions and solve decision-based problems	Students can construct and evaluate logical expressions	CO1, CO2	Apply, Analyze	Output prediction exercise	R1
8	I	Operators and Expressions	Bitwise and Special Operators	Understand and apply bitwise and special operators	Interactive Lecture + Demonstration	Perform basic bitwise operations on integer values and analyze results	Students can explain and implement basic bitwise operations	CO2	Understand, Apply	Short quiz + coding task	R1, R2
9	I	Operators and Expressions	Precedence and Associativity	Understand the order of evaluation of operators	Board Work + Code Tracing	Give students complex expressions and ask them to predict results before execution	Students can evaluate expressions using precedence and associativity rules	CO1, CO2	Analyze	Expression-tracing assessment	R1
10	I	Operators and Expressions	IEEE Representation, Lvalue and Rvalue	Understand representation of float and double, and distinguish Lvalue and Rvalue	Conceptual Lecture + Visualization	Analyze sample expressions and classify operands as Lvalue or Rvalue	Students can explain basic representation concepts and distinguish Lvalue/Rvalue	CO2	Understand, Analyze	Conceptual quiz	R1, R2
11	I	Type Conversion	Type Promotion in Expressions	Understand implicit type promotion during expression evaluation	Lecture + Demonstration	Trace expressions involving different data types and predict the resulting type/value	Students can explain type promotion in C expressions	CO2	Understand, Analyze	Code-tracing exercise	R1
12	I	Type Conversion	Assignment Conversion, Truncation and Casting	Apply explicit and implicit type conversion appropriately	Live Coding + Problem Solving	Compare results of implicit conversion and explicit casting using numerical examples	Students can use type casting and identify truncation effects	CO1, CO2	Apply, Analyze	Programming exercise	R1, R3
13	I	Decision and Case Control	if and if-else	Implement basic decision-making using conditional statements	Lecture + Live Coding	Develop programs for positive/negative, even/odd and eligibility-type problems	Students can implement basic conditional logic	CO1, CO2	Apply	Coding assessment	R1, R3
14	I	Decision and Case Control	Nested if-else	Implement multiple levels of decision making	Problem-Based Learning	Solve problems such as finding the largest of three numbers	Students can construct nested conditional solutions	CO1, CO2	Apply, Analyze	Problem-solving exercise	R1
15	I	Decision and Case Control	switch Statement	Use switch for multi-way decision making	Lecture + Demonstration	Develop a menu-driven calculator using switch	Students can implement menu-driven programs using switch	CO2	Apply	Programming task	R3
16	I	Decision and Case Control	switch versus if-else ladder	Select an appropriate decision structure for a given problem	Comparative Discussion + Problem Solving	Solve the same problem using both switch and if-else and compare approaches	Students can distinguish appropriate use cases for switch and if-else	CO1, CO2	Analyze, Evaluate	Comparative coding exercise	R1, R3
17	I	Loop Control Structure	for Loop	Understand and implement definite repetition using for	Live Coding + Guided Practice	Generate multiplication tables, sums and numerical sequences	Students can design programs using for loops	CO1, CO2	Apply	Coding exercise	R1
18	I	Loop Control Structure	while and do-while	Differentiate and implement entry-controlled and exit-controlled loops	Demonstration + Problem Solving	Implement programs using both while and do-while and compare execution	Students can select and implement suitable loop structures	CO1, CO2	Apply, Analyze	Code-tracing + coding task	R1, R3

Lec. No.	Mod.	Topic	Sub-topic	Learning Objective	Teaching Method	Proposed Activity	Expected Outcome	CO	Bloom's	Assessment	Ref.
19	I	Loop Control Structure	Nested Loops, break, continue and goto	Use nested loops and loop-control statements	Live Coding + Problem-Based Learning	Develop pattern-printing programs and demonstrate break and continue	Students can control loop execution and implement nested loops	CO1, CO2	Apply, Analyze	Programming exercise	R1
20	I	Arrays	1-D Arrays and Operations	Understand array declaration, initialization, address calculation and basic operations	Lecture + Live Coding	Implement insertion, deletion and traversal in a one-dimensional array	Students can declare, initialize and manipulate 1-D arrays	CO3	Apply, Analyze	Coding assessment	R1, R3
21	II	Two-Dimensional Arrays	Declaration, Initialization and Accessing Elements	Understand the representation and manipulation of 2-D arrays	Lecture + Demonstration	Create and display a 2-D array and access individual elements	Students can declare, initialize and access elements of 2-D arrays	CO3	Understand, Apply	Coding exercise	R1
22	II	Two-Dimensional Arrays	Applications and Problem Solving	Apply 2-D arrays to computational problems	Problem-Based Learning	Solve a problem using rows, columns and matrix-style data	Students can apply 2-D arrays to solve computational problems	CO3, CO8	Apply, Analyze	Problem-solving assessment	R1, R3
23	II	Matrix Operations	Matrix Addition and Subtraction	Implement addition and subtraction of matrices	Live Coding + Guided Practice	Implement matrix addition and subtraction for compatible matrices	Students can develop programs for basic matrix operations	CO3	Apply	Programming assignment	R1
24	II	Matrix Operations	Matrix Multiplication	Understand the logic and implementation of matrix multiplication	Step-by-Step Problem Solving	Derive the multiplication algorithm manually and implement it in C	Students can implement matrix multiplication using nested loops	CO3, CO8	Apply, Analyze	Coding assessment	R1, R3
25	II	Matrix Operations	Matrix Transposition	Understand and implement matrix transpose	Demonstration + Coding	Convert a matrix into its transpose and verify the result	Students can implement matrix transposition using 2-D arrays	CO3	Apply	Programming exercise	R1
26	II	Strings	Introduction to Strings	Understand strings as character arrays and their declaration/initialization	Interactive Lecture + Demonstration	Create character arrays and string variables and examine their memory representation	Students can declare and initialize strings in C	CO3	Understand, Apply	Short quiz + coding task	R1, R3
27	II	String Operations	String Length and Copy	Implement string length and copy operations with and without built-in functions	Live Coding + Guided Practice	Implement length and copy operations manually and using library functions	Students can perform string length and copy operations	CO3	Apply	Coding exercise	R1
28	II	String Operations	Reverse and Concatenate	Implement string reversal and concatenation	Problem-Based Learning + Live Coding	Develop programs to reverse a string and concatenate two strings without built-in functions	Students can manipulate strings using character-level operations	CO3, CO8	Apply, Analyze	Programming assignment	R1, R3
29	II	String Operations	String Comparison	Compare strings with and without built-in functions	Demonstration + Code Tracing	Compare manual character-by-character comparison with library-based comparison	Students can compare strings using different approaches	CO3	Apply, Analyze	Coding exercise	R1
30	II	String	1-D and 2-D Character Arrays	Understand declaration, initialization and use of collections of strings	Lecture + Coding Demonstration	Store and display names/words using 1-D and 2-D character arrays	Students can work with character arrays and collections of strings	CO3	Apply	Programming exercise	R1, R3
31	II	Functions	Function Declaration and Definition	Understand the need for functions and create user-defined functions	Interactive Lecture + Live Coding	Convert a large program into smaller functions	Students can declare, define and call functions	CO4	Understand, Apply	Coding assessment	R1, R3
32	II	Functions	Categories of Functions	Understand different categories of functions based on parameters and return values	Lecture + Demonstration	Implement examples representing different function categories	Students can classify and implement different categories of functions	CO4	Understand, Apply	Short quiz + coding task	R1
33	II	Functions	Parameter Passing — Call by Value	Understand parameter passing using call by value	Code Tracing + Demonstration	Trace changes to formal and actual parameters during function calls	Students can explain and implement call-by-value functions	CO4	Understand, Analyze	Code-tracing assessment	R1, R3

Lec. No.	Mod.	Topic	Sub-topic	Learning Objective	Teaching Method	Proposed Activity	Expected Outcome	CO	Bloom's	Assessment	Ref.
34	II	Functions	Passing Arrays to Functions	Pass arrays to functions and process array data modularly	Live Coding + Problem Solving	Create functions for array operations such as search, sum and sorting	Students can pass arrays to functions and modularize array-based solutions	CO3, CO4	Apply, Analyze	Programming assignment	R1
35	II	Pointers	Pointer Declaration and Initialization	Understand pointer variables, addresses and pointer initialization	Conceptual Lecture + Memory Visualization	Draw memory diagrams showing variables, addresses and pointers	Students can declare and initialize pointer variables	CO6	Understand, Apply	Conceptual quiz	R1, R3
36	II	Pointers	Accessing Variables through Pointers	Understand dereferencing and accessing variables through pointers	Demonstration + Live Coding	Write a program using address-of and dereference operators	Students can access and modify variables through pointers	CO6	Apply	Coding exercise	R1
37	II	Pointers	Arrays, Strings and Pointer Arithmetic	Understand the relationship between arrays, strings and pointers	Visualization + Problem Solving	Traverse an array/string using pointer arithmetic instead of indexing	Students can manipulate arrays and strings using pointers	CO3, CO6	Apply, Analyze	Programming assessment	R1, R3
38	II	Pointers	Pointers to Pointers, Array of Pointers, Pointer to Array, 2-D Arrays and Pointers	Understand advanced pointer relationships and applications	Conceptual Lecture + Code Tracing	Construct memory diagrams and trace advanced pointer expressions	Students can explain and implement advanced pointer structures	CO6	Analyze	Code-tracing + quiz	R1, R2
39	II	Pointers	Function Pointers, Dynamic Memory Allocation, void and NULL Pointers	Understand advanced pointer applications and dynamic memory management	Demonstration + Guided Coding	Demonstrate dynamic allocation and safe use of pointers	Students can explain and apply dynamic memory allocation and advanced pointer concepts	CO6	Apply, Analyze	Coding assessment	R1, R3
40	II	User Defined Types	enum, typedef, Union and Structure	Understand and apply user-defined data types	Lecture + Integrated Problem Solving	Design a simple student/employee record using appropriate user-defined types	Students can define and use enum, typedef, union and structure concepts	CO6, CO8	Understand, Apply, Create	Integrated coding assessment	R1, R3

5. Reference Key

Code	Reference
R1	K. N. King, "C Programming: A Modern Approach", 2nd Edition, W. W. Norton & Company.
R2	K. R. Venugopal & Sudeep R. Prasad, "Mastering C".
R3	Paul Deitel & Harvey Deitel, "C How to Program", 8th Edition, Pearson.
R4	Peter van der Linden, "Expert C Programming: Deep C Secrets", Prentice Hall.
R5	W. Richard Stevens, "Advanced Programming in the UNIX Environment", 3rd Edition, Addison-Wesley Professional.

6. Note on CO Coverage

The supplied curriculum lists CO5 (recursion) and CO7 (file handling) among the Course Outcomes; however, recursion and file handling are not included in the detailed 40-hour Module I/II content. Accordingly, this LDP does not artificially assign separate lectures to those topics. If required by the department, they may be incorporated later as enrichment topics or after verification of the applicable revised curriculum.

Placement-aligned Action Plan/Value Addition

Objective: To enhance the employability and industry readiness of students by facilitating an in-depth understanding of leading companies in the computer science domain and familiarizing them with placement-focused interview preparation, grounded in core C programming skills.

Key Activities:

1. Company Case Studies:

- Each group of 3-4 students will conduct a comprehensive case study on a prominent company in the computer science/IT domain.
- Work Culture: Analysis of the company's internal environment, values, and professional practices.
- Employment Procedure: Detailed overview of the hiring process, including coding rounds, interview formats, and selection criteria.
- Domains of Expertise: Identification of key technical skills required (C programming fundamentals, DSA using C, problem-solving).
- Tools and Technologies: Examination of compilers, IDEs, and technologies predominantly used by the company.
- World Ranking and Recognition: Evaluation of the company's position in global rankings and its influence in the tech industry.

2. Placement Question Pool:

- A curated collection of C programming questions commonly asked during placement drives and campus interviews.
- Technical Questions: Array manipulation, string handling, pointers, recursion, and output-prediction/code-tracing questions.
- Situational/Problem-solving Questions: Addressing real-world scenarios to gauge problem-solving abilities and logical thinking.
- Random viva will be conducted using this question pool to prepare students for actual placement interviews.

Scenario-Based Sample Assignment Sheets

Assignment-1 (Module-1)

Course: B.Tech (CSE) Year: I Semester: I Session: _____

Subject Code & Name: BCSG 0802 (Computer Programming)

1. A milk dairy wants to compute the total bill for a customer based on the quantity of milk purchased (in litres) and the price per litre. Write a C program that reads the quantity and rate, and prints the total amount using appropriate formatted input/output.
2. A logistics company needs to classify parcels as 'Light', 'Medium', or 'Heavy' based on their weight in kilograms (Light: <5kg, Medium: 5-20kg, Heavy: >20kg). Write a C program using if-else ladder to classify a parcel and print the category.
3. An examination portal calculates a student's grade based on marks obtained out of 100 using a grading scale (A: ≥ 90 , B: 75-89, C: 60-74, D: 40-59, F: <40). Implement this using a switch-case structure on the tens digit of the marks, handling all edge cases.
4. A weather monitoring station records the temperature (in Celsius) every hour for 24 hours. Write a C program using a one-dimensional array to store the readings and compute the average, maximum, and minimum temperature of the day.
5. A retail store maintains a list of product prices in an array. Design a C program that performs a Linear Search to check whether a given price exists in the list, and if so, prints its position.
6. An online quiz platform stores students' scores in an array and needs the scores sorted in ascending order for ranking. Implement Bubble Sort in C to sort the array of scores and display the sorted list.
7. A library management system needs to insert a new book's accession number into a sorted array at the correct position, and also needs to delete a book's record when withdrawn. Write a C program implementing insertion and deletion operations on a 1D array.
8. A ticket-vending machine accepts only 4-digit binary access codes represented as an integer (0s and 1s). Write a C program that reads a number and, using loop and conditional constructs, verifies whether every digit of the number is either 0 or 1.
9. A fitness tracking app calculates the number of calories burnt using a formula involving type conversion between float and int values (e.g., converting user-entered height in cm as float to an int for a lookup table). Write a C program demonstrating explicit type casting and explain the truncation that occurs.
10. A parking lot management system needs to determine whether a floor number entered by an attendant is valid (between 1 and 10). Use nested if-else to validate the floor number and also check whether it is odd or even for zone allocation.

11. A number-guessing game needs to print the multiplication table of a number entered by the user, using a for loop, and should stop early using a break statement if the product exceeds 100.
12. A bank's ATM simulation reads a sequence of transaction amounts (terminated by -1) and computes the running balance using a while loop, rejecting (using continue) any transaction amount that is negative and not equal to -1.

Assignment-2 (Module-2)

Course: B.Tech (CSE) Year: I Semester: I Session: _____

Subject Code & Name: BCSG 0802 (Computer Programming)

1. A seating arrangement system for an examination hall stores seat allocations for 5 rows and 6 columns of a room using a 2D array. Write a C program to initialize, display, and search for a given roll number placed in a specific seat.
2. An institute wants to compute the total marks of students across 3 subjects using matrix addition, where two 2D arrays represent internal and external marks. Write a C program to add the two matrices and display the resultant total-marks matrix.
3. A robotics company represents sensor calibration data as two matrices and needs their product for a transformation computation. Write a C program to perform matrix multiplication for compatible matrices and handle the case where multiplication is not possible.
4. A image-processing utility needs to transpose a matrix representing pixel intensity values. Write a C program to compute and display the transpose of a given 2D matrix.
5. A password-strength checker needs to verify whether a user's password (a string) is a palindrome, without using any built-in string-reversal function. Write a C program to check this.
6. A text-formatting tool needs to count the length of a sentence, reverse it, and check whether two entered strings are identical — all without using built-in string library functions. Write a C program implementing these three operations from scratch.
7. A contact-management app allows searching a name in a list of names stored as an array of strings, using strcmp(). Write a C program that performs this search using built-in string functions and reports whether the name was found.
8. A calculator utility needs a reusable function to compute the factorial of a number, invoked multiple times for different menu options. Write a C program using Call by Value to implement and repeatedly call a factorial function.
9. A statistics tool needs to compute the sum and average of elements in an array by passing the array to a function. Write a C program that passes an array to a function and returns the sum and average using appropriate techniques.
10. A banking application uses pointers to directly update an account balance passed to a function, rather than returning a value. Write a C program demonstrating how a pointer to a variable is used to modify its value inside a function.
11. A memory-diagnostics tool for embedded systems needs to demonstrate pointer arithmetic by traversing an array purely through a pointer (not indexing). Write a C program that prints all elements of an array using pointer arithmetic only.
12. A hospital's patient-record system stores each patient's ID, name, and age using a structure. Write a C program that defines a structure Patient, creates an array of 5 patients, and displays records of patients above a given age using the structure.
13. A company's employee database needs to distinguish between full-time and part-time employees using minimal memory, where only one of several fields (hourly rate or annual salary) is relevant at a time. Write a C program using a union to model this, and explain why a union is more memory-efficient than a structure here.
14. An inventory system uses typedef to define a custom type for product codes, and an enum to represent order status (PENDING, SHIPPED, DELIVERED, CANCELLED). Write a C program that defines these and uses them to print the status of a sample order.

Sample Tutorial Sheets / Practice Sheets

Tutorial Sheet (Module-1)

Course: B.Tech (CSE) Year/Semester: 1st / I Session: _____

Subject Name & Code: Computer Programming (BCSG 0802)

Ques. Based on Basics of C, Operators & Type Conversion:

1. Write a C program to print 'Hello, World!' and explain the role of the main() function and header files.

2. Write a C program to demonstrate the use of int, float, char, and double data types by declaring and printing one variable of each.
3. Write a C program to swap two numbers using a temporary variable, and then without using a temporary variable.
4. Write a C program to demonstrate the use of all arithmetic operators (+, -, *, /, %) on two integers.
5. Write a C program to demonstrate the difference between pre-increment and post-increment operators with a suitable example.
6. Write a C program to evaluate the precedence and associativity of the expression: $a + b * c - d / e$.
7. Explain Lvalue and Rvalue with suitable C code examples.
8. Write a C program to demonstrate implicit type conversion (type promotion) when adding an int and a float.
9. Write a C program to demonstrate explicit type casting by converting a float value to int and explain the truncation.

Ques. Based on Decision and Loop Control Structures:

1. Write a C program to check whether a given number is positive, negative, or zero using if-else.
2. Write a C program to find the largest of three numbers using nested if-else.
3. Write a C program to check whether a year is a leap year.
4. Write a C program to build a simple calculator (add, subtract, multiply, divide) using switch-case.
5. Write a C program to print the Fibonacci series up to n terms using a for loop.
6. Write a C program to check whether a given number is prime using a while loop.
7. Write a C program to reverse the digits of a number using a do-while loop.
8. Write a C program to print all prime numbers between 1 and 100 using nested loops.
9. Write a C program to demonstrate the use of break and continue inside a loop.
10. Write a C program to print a right-angled triangle pattern of stars using nested loops.

Ques. Based on Arrays (1D):

1. Write a C program to find the largest and smallest element in a 1D array.
2. Write a C program to calculate the sum and average of elements of an array.
3. Write a C program to reverse the elements of an array without using a second array.
4. Write a C program to count the number of even and odd elements in an array.
5. Write a C program to insert an element at a given position in an array.
6. Write a C program to delete an element from a given position in an array.
7. Write a C program to perform Linear Search on an array of n elements.
8. Write a C program to sort an array using Bubble Sort and display the number of swaps performed.
9. Write a C program to find the second-largest element in an array without sorting it.
10. Write a C program to merge two arrays into a third array.

Tutorial Sheet (Module-2)

Course: B.Tech (CSE) Year/Semester: 1st / I Session: _____

Subject Name & Code: Computer Programming (BCSG 0802)

Ques. Based on 2D Arrays & Matrix Operations:

1. Write a C program to read and display a 2D array (matrix) of order m x n.
2. Write a C program to add two matrices of the same order.
3. Write a C program to subtract two matrices of the same order.

4. Write a C program to multiply two matrices where the number of columns of the first equals the number of rows of the second.
5. Write a C program to find the transpose of a given matrix.
6. Write a C program to check whether a given square matrix is symmetric.
7. Write a C program to find the sum of the diagonal elements of a square matrix.

Ques. Based on Strings:

1. Write a C program to find the length of a string without using `strlen()`.
2. Write a C program to copy one string into another without using `strcpy()`.
3. Write a C program to reverse a string without using `strrev()`.
4. Write a C program to concatenate two strings without using `strcat()`.
5. Write a C program to compare two strings without using `strcmp()`.
6. Write a C program to check whether a given string is a palindrome.
7. Write a C program to count the number of vowels and consonants in a string.
8. Write a C program to count the occurrences of a given character in a string.

Ques. Based on Functions:

1. Write a C program with a function to check whether a number is prime, called from `main()` for a range of numbers.
2. Write a C program to find the factorial of a number using a function (Call by Value).
3. Write a C program with a function that accepts an array and its size, and returns the sum of its elements.
4. Write a C program with a function to find the maximum element of an array passed as a parameter.
5. Explain, with a program, the difference between actual parameters and formal parameters using Call by Value.

Ques. Based on Pointers:

1. Write a C program to demonstrate the declaration, initialization, and dereferencing of a pointer variable.
2. Write a C program to access and modify the value of a variable using a pointer.
3. Write a C program to demonstrate pointer arithmetic by traversing an array using a pointer.
4. Write a C program to demonstrate the relationship between arrays, strings, and pointers.
5. Write a C program to demonstrate the use of a pointer to a pointer (double pointer).
6. Write a C program to demonstrate an array of pointers used to store multiple strings.
7. Write a C program to demonstrate dynamic memory allocation using `malloc()` and `calloc()`, and free the allocated memory.
8. Write a C program to demonstrate the use of a void pointer and explain why it cannot be dereferenced directly.
9. Write a C program to demonstrate the difference between NULL pointer and an uninitialized pointer.
10. Write a C program to demonstrate the use of a pointer to a function.

Ques. Based on User-Defined Types (enum, typedef, union, structure):

1. Write a C program to define a structure Student with roll number, name, and marks, and display details for 3 students.
2. Write a C program to define a structure Point with x and y coordinates and write a function to calculate the distance between two points.
3. Write a C program to demonstrate the use of typedef to create an alias for a structure type.
4. Write a C program to demonstrate the use of a union to store either an integer or a float in the same memory location, and explain the memory savings compared to a structure.
5. Write a C program to define an enum for days of the week and print the day name corresponding to a given number.

Minutes of Meetings

DEPARTMENT OF COMPUTER ENGINEERING & APPLICATIONS

GLA UNIVERSITY, MATHURA

Dear Colleague,

Please find below the Minutes of Meeting (MOM) for the subject Computer Programming (BCSG 0802) in B.Tech I Year, held on _____.

Present Members:

ACTION POINTS

S. No.	Action Point Description	Responsible Person	Due Date
1	Industry-Oriented Assignments (Minimum 15 questions each)		
2	Standard Quizzes (Minimum 15-20 questions each)		
3	PPT for Introductory Lectures		
4	Preparation of Course File		
5	Assessment Calendar		
6	Updation of LDP [Detailed]		

DISCUSSION POINTS

- Marking scheme: Mid Term: 30, End Term: 45, TA (20), Attendance (5).
- All faculty members are required to conduct their classes as per the scheduled timetable.
- All members are advised not to avail leave during the semester, except in unavoidable circumstances.

Regards,

Mr. Ajay Ashok Dabade.

SME, Computer Programming

Previous Year Question Papers

To be added.